Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

ALGORITHMIC MAZE SIMULATOR

Team Name: Team #	*HUSTLERS* *DSCPP-III-2026-2338*
Team member 1 (Team Lead)	*Jaydeep sharma – 2027835* *2510010920@geu.ac.in* 
Team member 2	*Jaskirat singh – 2027834* *2510010581@geu.ac.in* 
	member .png
	member .png

PHASE-II PROJECT PROGRESS DESCRIPTION (35 pts)

Project Summary

Making and solving a maze shows how computer methods can solve path-finding problems. Making and solving mazes by hand can take a lot of time and may not be a quick way to test paths or maze levels.

Our project called Maze Generator and Solver creates new mazes and finds a path from the start point to the goal. It uses data structures and algorithms (DSA) such as grids stacks queues DFS BFS recursion and backtracking to create and solve

mazes.

The project also uses object-oriented programming (OOP) ideas such as classes and objects along with encapsulation abstraction and a clear design to organise the parts of the program.

This is useful because maze-solving methods are linked to real path-finding and navigation tasks such as robot movement games route planning and artificial intelligence.

This project will help us learn how DSA methods work in real uses and how OOP can help us build software that is easy to manage and update.

Updated Project Design and Working

Our project has two parts. A C++ program does the main work and a web page makes it easy to see and use.

In C++ we used object-oriented design. The Maze class stores a grid of Cell objects and MazeGenerator makes a random maze using DFS and backtracking. We made a base class called Solver and DFSSolver and BFSSolver use it and each has its own solve() function. DFS uses a stack so it follows one path first. BFS uses a queue and checks cells level by level. This means it finds the shortest path. Both use parent pointers to trace the path back. Game handles manual play. Visualizer shows the maze in the console and HtmlExporter saves the maze results in index.html. Common types such as CellState and Position are kept in Common.h so all files use the same definitions.

The web page uses HTML CSS and JavaScript. Users can choose a level and create a maze play with the keyboard or watch a solver search. A speed slider and live stats show cells checked path length time taken and player moves. Different colours show the start the goal visited cells and final path so the maze is easy to follow.

We build the project with build.bat and g++ (C++14). This creates maze_game.exe. We open the web page with Live Server in VS Code. We use libraries such as iostream vector string chrono and queue.

Tasks Completed

Task Completed	Team Member
1.Created the C++ project structure with separate classes for maze cells generator solvers and visualizer.	Jaydeep sharma
2. Built a maze generator using DFS (recursive backtracking) with 5 levels: 11x11 15x15 and 21x21 31x3141x41	*Jaydeep sharma*
3. Added the DFS solver (stack) and BFS solver (queue) plus a test to compare them.	*Jaydeep sharma*
4. Added manual play mode with W/A/S/D and arrow keys.	*Jaskirat singh*
5.Designed the HTML/CSS interface with colour-coded cells speed slider and live stats (nodes explored path length time moves).	*Jaskirat singh*

Problems Faced

While making the game we had some problems keeping it smooth and easy to play. One challenge was making sure the player moved correctly and the controls worked well. We also had errors when checking if the player reached the goal. We will test the game often and fix errors when they appear. Another challenge is making the game screen easy to understand. We will use clear instructions simple controls and different colours to make the game fun and easy to use.

Tasks Left

Task Pending	Team Member (to complete the task)
Add a stepwise setup flow: welcome screen difficulty selection and custom maze size with start and goal positions.	*Jaydeep sharma*
2. Build a manual maze editor to add or remove walls with a check that the goal is can be reached.	*Jaydeep sharma*
3. Add three more algorithms: A* Dijkstra and Greedy Best-First Search using priority queues.	*Jaydeep sharma*
4. Show the search live on the maze with pause resume reset and speed controls plus live stats.	*Jaskirat singh*
5. Improve the test to compare all algorithms with a table and bar chart.	*Jaskirat singh*
6. Add a local top-5 leaderboard saved in the browser then do final testing bug fixing and report writing.	*Jaskirat singh*

Project Result/Work to Submit

This game helps players learn two ways to solve a maze. BFS checks nearby paths first and finds the shortest way to the goal. DFS follows one path as far as it can then goes back to try another path. This shows that a problem can have more than one solution.

The game also teaches patience and planning. When players reach a dead end they learn to go back and try another way instead of giving up. It helps them focus and think before making a move.

A maze game is a fun way to learn problem solving. It helps players think clearly and make better choices and keep trying until they reach the goal. These methods are also used in robots and games and navigation systems.

Progress Summary

Our maze game is about 70% complete. Most main features are ready and the game is working well. We are now improving it and adding new features to make it more fun for players.

We are making a feature that lets players create their own maze levels. This is taking longer than planned because we want it to work well and be easy to use. Because of this we are a little late. We are still working on it and hope to finish it soon.

We also plan to add more ways to solve the maze. This will give players more ways to find the path to the goal and see how each method works. We want users to compare different ways to solve the same maze.

Codebase Information

https://github.com/sen339853-rgb/ALGORITHMIC-MAZE-SIMULATOR-.git

Branch: CS
Important commits:

- Initial project structure with Maze Cell and Solver classes
- Added DFS maze generator and BFS/DFS solvers

Testing Status

Test Type	Status (Pass/Fail)	Notes
1. Compilation (build.bat)	Pass	Builds without errors and creates maze_game.exe
2. Maze generation 3 levels	Pass	11x11 15x15 and 21x21 mazes are generated
3. BFS shortest path	Pass	Level 1 solved in 25 steps and shown on the grid
4. DFS solver	Pass	Finds a valid path not always the shortest
5. Manual play movement	Pass	Controls work some goal-detection errors being fixed

Work to Submit Progress

About 70% of our maze game is complete and we have a working version to show. The BFS and DFS solvers are ready and working. Players can use them to find the path to the goal. We are making a feature that lets players create their own maze levels. This is still in progress and needs more time. We also need to make some final changes such as improving the look fixing small errors and making the controls work well.
Overall the project is going well and we are working to finish the remaining tasks soon.